1. Cyber Forensics AI – Neural Attack Reconstruction

Tech Stack: Python, FastAPI, VAE-GAN, Docker, Machine Learning

Project Overview

Cyber Forensics AI is an intelligent cybersecurity investigation system designed to reconstruct missing or corrupted attack logs using Generative AI. In real-world cyberattacks, attackers often delete traces of their activities, making forensic investigations difficult. This project addresses that problem by predicting and recreating missing attack events.

Problem Statement

Security teams frequently encounter incomplete attack logs due to:

- Log corruption
- Malicious deletion
- System failures
- Network interruptions

Incomplete logs make it difficult to identify attack patterns and understand the attack timeline.

Solution

I developed a hybrid Generative AI architecture combining:

- Variational Autoencoders (VAE)
- Generative Adversarial Networks (GAN)

The VAE learns the latent representation of attack patterns, while the GAN generates realistic missing log entries.

My Responsibilities

- Collected and preprocessed cybersecurity datasets.
- Designed and trained VAE-GAN architecture.
- Built FastAPI backend APIs for inference.
- Implemented confidence scoring mechanism.
- Containerized the entire application using Docker.
- Optimized inference pipelines.

Key Achievements

- Improved attack traceability significantly.
- Reduced inference latency by approximately 30%.
- Enabled real-time forensic reconstruction.
- Created scalable deployment architecture.

Interview Explanation

"The project uses Generative AI to reconstruct missing cyberattack logs. Since attackers often remove evidence from systems, I trained a VAE-GAN model to learn attack patterns and generate realistic missing events. I deployed the model using FastAPI and Docker, reducing inference latency by around 30% while improving attack traceability."

---

2. AI Text Humanizer

Tech Stack: Flask, LangChain, GPT-4o, Supabase, NLP

Project Overview

AI Text Humanizer is a SaaS platform that converts AI-generated text into more natural, human-like writing while preserving the original meaning.

Problem Statement

Many AI-generated texts:

- Sound robotic
- Contain repetitive patterns
- Are easily detectable by AI detectors
- Lack human tone and emotional variation

Solution

I built a Retrieval-Augmented Generation (RAG) system using LangChain and GPT models to rewrite text in multiple tones such as:

- Professional
- Friendly
- Academic
- Persuasive
- Creative

Features

- User authentication
- Session management
- Tone selection
- Humanization scoring
- AI detection reduction
- History management

My Contributions

- Designed Flask backend.
- Integrated GPT-4o APIs.
- Built LangChain workflows.
- Developed Supabase authentication.
- Optimized prompt engineering.
- Improved response quality through evaluation testing.

Results

- Improved text naturalness by 35%.
- Maintained response latency below 200 ms.
- Successfully handled concurrent users.

Interview Explanation

"I built an AI Text Humanizer that transforms AI-generated content into more natural and engaging text. Using LangChain, GPT-4o, Flask, and Supabase, I created a RAG-based architecture capable of rewriting content in multiple tones while maintaining semantic meaning."

---

3. Stock Price Analyzer

Tech Stack: LSTM, Flask, React, Pandas, NumPy

Project Overview

A machine learning-powered stock market prediction platform that forecasts future stock price movements using historical market data.

Problem Statement

Investors often struggle to identify trends and make data-driven investment decisions.

Solution

I developed a Long Short-Term Memory (LSTM) neural network capable of learning temporal dependencies from stock market data.

Workflow

1. Collect stock data from Yahoo Finance.
2. Clean and preprocess data.
3. Create time-series sequences.
4. Train LSTM model.
5. Predict future trends.
6. Visualize predictions.

My Contributions

- Built data pipeline.
- Performed feature engineering.
- Developed LSTM architecture.
- Created Flask APIs.
- Built React frontend dashboard.

Results

- Achieved approximately 62% directional accuracy.
- Real-time stock visualization dashboard.
- End-to-end ML deployment.

Interview Explanation

"I developed an LSTM-based stock prediction system that learns patterns from historical stock market data. The application combines machine learning, data visualization, and full-stack deployment using Flask and React."

---

4. Enhanced Face and Eye Detection System

Tech Stack: OpenCV, Haar Cascades, Flask, Computer Vision

Project Overview

A real-time computer vision application capable of detecting faces and eyes from live webcam feeds.

Problem Statement

Many vision systems struggle under:

- Different lighting conditions
- Occlusions
- Real-time performance requirements

Solution

I built an optimized detection pipeline using OpenCV Haar Cascade classifiers.

Features

- Real-time webcam detection
- Face localization
- Eye localization
- FPS monitoring
- Flask-based interface

My Contributions

- Implemented OpenCV pipeline.
- Optimized frame processing.
- Developed Flask dashboard.
- Improved detection stability.

Results

- Achieved 95%+ detection accuracy.
- Maintained approximately 30 FPS.
- Worked under varying lighting conditions.

Interview Explanation

"This project focuses on real-time face and eye detection using OpenCV Haar Cascades. I optimized the processing pipeline to maintain over 30 FPS while achieving more than 95% detection accuracy."

---

5. Student Portal Management System

Tech Stack: Flask, PostgreSQL, REST APIs

Project Overview

A web-based student management platform that handles academic records and administrative operations.

Problem Statement

Manual student record management is inefficient and prone to errors.

Solution

Developed a role-based web application supporting:

- Admins
- Faculty
- Students

Features

- Authentication
- CRUD operations
- Student records management
- REST APIs
- Database management

My Contributions

- Designed PostgreSQL schema.
- Built Flask backend.
- Developed REST APIs.
- Implemented role-based access control.
- Optimized database queries.

Results

- Managed 2500+ student records.
- Improved query performance by 40%.
- Reduced manual administrative effort.

Interview Explanation

"I developed a Student Portal Management System using Flask and PostgreSQL that manages over 2500 student records. The system includes role-based authentication, REST APIs, and optimized database operations."